INSTITUTE OF ENGINEERING & MANAGEMENT, GURUKUL

PCCCS495 – Advanced Programming

Student Attendance Management System

Submitted by: Soumik Mondal

Roll Number: 55

Enrollment Number:- 12024052019086

1. Problem Statement

Managing student attendance manually is time-consuming and error-prone. Educational institutions need a reliable system to track student presence across multiple subjects, generate reports, and alert teachers when attendance falls below acceptable levels.

This project presents a Student Attendance Management System built in Java using Object-Oriented Design principles. The system allows teachers to add students, mark attendance subject-wise, generate attendance reports by student or subject, and automatically alert when a student's attendance drops below 75%. Data is persisted across sessions using Java serialization.

The system is built with a clean MVC architecture, implementing the Observer and Strategy design patterns, demonstrating strong OOP principles including abstraction, inheritance, polymorphism, and encapsulation.

---

2. System Design

2.1 Package Structure

```
com.attendance/
    model/       - Data classes (Person, Student, Teacher, Subject, AttendanceRecord)
    exception/   - Custom exceptions
    observer/    - Observer pattern implementation
    strategy/    - Strategy pattern implementation
    repository/  - File persistence layer
    service/     - Business logic
    ui/          - Swing GUI components
    controller/  - MVC Controller
```

2.2 Major Classes

Person (abstract) - Base class for all people in the system. Contains common fields id and name with getters and setters. Declares abstract method getInfo() forcing subclasses to provide their own implementation.

Student - Extends Person. Adds rollNumber and department fields. Implements getInfo() to return student-specific information.

Teacher - Extends Person. Adds employeeId and department. Implements getInfo() for teacher-specific information.

Subject - Represents a subject with code, name, and total classes held.

AttendanceRecord - Stores a single attendance entry with studentId, subjectCode, date, and present status.

AttendanceService - The core service class. Manages students, subjects, and records. Coordinates observers and strategies. Handles all business logic.

MainController - MVC Controller connecting the GUI to the service layer.

AlertObserver - Implements AttendanceObserver. Fires an alert when attendance drops below the configured threshold (default 75%).

StudentReportStrategy - Implements ReportStrategy. Generates report grouped by student.

SubjectReportStrategy - Implements ReportStrategy. Generates report grouped by subject.

StudentRepository - Implements DataRepository. Saves and loads Student objects using Java serialization.

AttendanceRepository - Implements DataRepository. Saves and loads AttendanceRecord
objects using Java serialization.

2.3 Interaction Flow

Teacher opens application -> Adds students via StudentPanel -> Marks attendance via AttendancePanel -> AttendanceService notifies AlertObserver if attendance is low -> Teacher generates report via ReportPanel -> ReportStrategy generates formatted output -> Data saved to .dat files automatically.

3. OOP Concepts Demonstrated

3.1 Abstraction

Where used: The Person class is declared abstract.

Why used: Person should never be instantiated directly. Only Student and Teacher are real entities. Abstraction hides implementation details and forces subclasses to implement getInfo().

Code snippet:

```
public abstract class Person implements java.io.Serializable {
    private String id;
    private String name;

    public Person(String id, String name) {
        this.id = id;
        this.name = name;
    }

    public abstract String getInfo();
}
```

Explanation: The abstract keyword on Person means no object of type Person can be created. The abstract method getInfo() acts as a contract — every subclass must provide its own implementation.

3.2 Inheritance

Where used: Student extends Person, Teacher extends Person.
Why used: Both Student and Teacher share common attributes (id, name). Inheritance avoids code duplication and creates a logical hierarchy.

Code snippet:

```
public class Student extends Person implements java.io.Serializable {
```

```
    private String rollNumber;
    private String department;

    public Student(String id, String name, String rollNumber, String department) {
        super(id, name);
        this.rollNumber = rollNumber;
        this.department = department;
    }
}
```

Explanation: The extends keyword establishes the inheritance relationship. super(id, name) calls the parent constructor, reusing Person's initialization logic.

3.3 Polymorphism

Where used: getInfo() is overridden in both Student and Teacher.
Why used: Different entities return different formatted information through the same method name. Also, ReportStrategy interface allows runtime switching between StudentReportStrategy and SubjectReportStrategy.

Code snippet:

```
@Override
public String getInfo() {
    return "Student[" + getRollNumber() + "] " + getName() + " - " + department;
}
```

Explanation: @Override ensures this method replaces the parent's abstract version. Calling getInfo() on any Person reference returns the correct subclass-specific output.

3.4 Encapsulation

Where used: All fields in every class are declared private.
Why used: Protects internal state from external modification. All access goes through controlled getters and setters.

Code snippet:

```
public class Subject {
    private String code;
    private String name;
    private int totalClasses;

    public String getCode() { return code; }
    public String getName() { return name; }
    public int getTotalClasses() { return totalClasses; }
}
```

Explanation: The private modifier ensures no external class can directly modify code, name, or totalClasses. Data integrity is maintained through controlled access.

3.5 Observer Pattern

Where used: AttendanceObserver interface, AlertObserver implementation, AttendanceService as the subject.
Why used: Decouples the attendance marking logic from the alerting logic. New observers can be added without modifying AttendanceService.

Code snippet:

```
public class AlertObserver implements AttendanceObserver {
    private double threshold;

    @Override
    public void update(String studentName, String subject, double percentage) {
        if (percentage < threshold) {
            System.out.println("ALERT: " + studentName +
                " attendance in " + subject + " is " + percentage + "%");
        }
    }
}
```

Explanation: When markAttendance() is called in AttendanceService, it calculates the updated percentage and notifies all registered observers. AlertObserver checks if the percentage is below threshold and fires an alert.

3.6 Strategy Pattern

Where used: ReportStrategy interface with StudentReportStrategy and SubjectReportStrategy implementations.
Why used: Allows the report generation algorithm to be selected at runtime without changing the service code.

Code snippet:

```
public interface ReportStrategy {
    String generate(List<Student> students, List<AttendanceRecord> records);
}

public String generateReport() {
```

```
    if (reportStrategy == null) return "No strategy selected.";
    return reportStrategy.generate(students, records);
}
```

Explanation: The service delegates report generation to whichever strategy is currently set. Switching from student report to subject report requires only one line: service.setReportStrategy(new SubjectReportStrategy()).

---

4. Implementation Highlights

4.1 File Persistence using Serialization

```
public void save(List<Student> students) {
    try (ObjectOutputStream oos = new ObjectOutputStream(
            new FileOutputStream(filePath))) {
        oos.writeObject(students);
    } catch (IOException e) {
```

```
        System.out.println("Error saving students: " + e.getMessage());
    }
}
```

Justification: ObjectOutputStream serializes the entire list to a binary file. The try-with-resources pattern ensures the stream is closed automatically even if an exception occurs. This provides persistent storage across application sessions.

4.2 Custom Exception Handling

```
public void markAttendance(String studentId, String subjectCode, boolean present) {
    Student found = null;
    for (Student s : students) {
        if (s.getId().equals(studentId)) { found = s; break; }
    }
    if (found == null) throw new StudentNotFoundException(studentId);
    if (subjectCode == null || subjectCode.isEmpty())
        throw new InvalidAttendanceException("Subject code cannot be empty");
}
```

Justification: Custom exceptions provide meaningful error messages instead of generic Java errors. This makes debugging easier and gives the GUI layer specific information to display to the user.

4.3 Observer Notification in Service

```
double percentage = total == 0 ? 0 : (presentCount * 100.0 / total);
for (AttendanceObserver observer : observers) {
    observer.update(found.getName(), subjectCode, percentage);
```

}

Justification: After every attendance mark, the service recalculates the attendance percentage and notifies all observers. This ensures alerts are triggered in real time without coupling the alert logic to the marking logic.

---

5. Testing and Error Handling

5.1 Test Cases

Test 1: Add Student
Input: ID=S001, Name=Debarghya, Roll=001, Dept=CSE
Expected: "Student added successfully"
Result: Pass

Test 2: Duplicate Student
Input: ID=S001 (already exists)

Expected: "Error: Student with ID S001 already exists"

Result: Pass

Test 3: Mark Attendance

Input: StudentID=S001, Subject=JAVA101, Present=true

Expected: "Attendance marked for student: S001"

Result: Pass

Test 4: Invalid Student ID

Input: StudentID=S999 (does not exist)

Expected: "Error: Student not found with ID: S999"

Result: Pass

Test 5: Generate Report

Input: Click Report by Student

Expected: Formatted report showing student attendance percentage

Result: Pass

5.2 Edge Cases Handled

- Empty fields validated before processing
- Student not found throws StudentNotFoundException
- Duplicate student entry throws DuplicateEntryException
- Empty subject code throws InvalidAttendanceException
- Missing data files handled gracefully with empty list return

---

6. Git Workflow

The project was developed incrementally over multiple days with meaningful commit messages following the conventional commits format.

Commit progression:

1. feat: initialize package structure for attendance management system
2. feat: add Person abstract base class with encapsulation
3. feat: add Student class extending Person (inheritance)
4. feat: add Teacher class extending Person (inheritance)
5. feat: add Subject model class
6. feat: add AttendanceRecord model class
7. feat: add custom exception classes for error handling

8. feat: implement Observer pattern for low attendance alerts
9. feat: implement Strategy pattern for report generation
10. feat: add file-based repository for data persistence
11. feat: add AttendanceService with Observer and Strategy integration
12. feat: add MainController connecting service to UI (MVC pattern)
13. feat: add Swing GUI - MainFrame, StudentPanel, AttendancePanel, ReportPanel
14. feat: add Main entry point to launch the application
15. docs: update README with project details

Each commit represents a logical unit of work, demonstrating incremental development and proper version control discipline.

---

7. Conclusion and Future Scope

7.1 Conclusion

The Student Attendance Management System successfully demonstrates core Object-Oriented Design principles in a real-world context. The system implements abstraction through the Person abstract class, inheritance through Student and Teacher subclasses, polymorphism through method overriding and interface-based design, and encapsulation through private fields and controlled access. Two design patterns — Observer and Strategy — elevate the architecture beyond basic CRUD operations. File serialization provides persistent data storage across sessions. The MVC architecture ensures clean separation between UI, business logic, and data layers.

7.2 Future Scope

- Export reports to PDF or Excel format
- Add login system with teacher authentication
- Implement database storage using JDBC instead of file serialization
- Add graphical charts showing attendance trends
- Send automated email alerts to students with low attendance
- Support multiple classrooms and batch management